

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
uploaded = files.upload()

video_path = list(uploaded.keys())[0]

print("Uploaded video:", video_path)
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
 Saving 5319441-uhd_2160_3840_25fps.mp4 to 5319441-uhd_2160_3840_25fps.mp4
 Uploaded video: 5319441-uhd_2160_3840_25fps.mp4

```
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    fps = cap.get(cv2.CAP_PROP_FPS)
    frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    print("Video opened successfully!")
    print("Width:", width)
    print("Height:", height)
    print("FPS:", fps)
    print("Total Frames:", frame_count)
```

Video opened successfully!
 Width: 1440
 Height: 2560
 FPS: 25.0
 Total Frames: 512

```
ret, frame1 = cap.read()
ret2, frame2 = cap.read()

if not ret or not ret2:
    print("Could not read video frames.")
else:
    print("Two consecutive frames read successfully.")

    frame1_rgb = cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB)
    frame2_rgb = cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.imshow(frame1_rgb)
    plt.title("Frame 1")
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.imshow(frame2_rgb)
    plt.title("Frame 2")
    plt.axis("off")

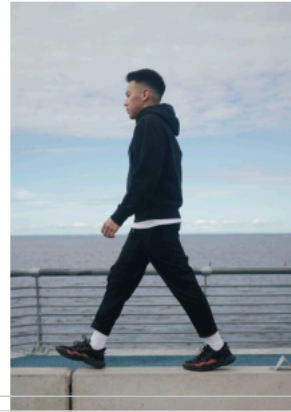
    plt.show()
```

Two consecutive frames read successfully.

Frame 1



Frame 2



```
gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.imshow(gray1, cmap='gray')
plt.title("Grayscale Frame 1")
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.imshow(gray2, cmap='gray')
plt.title("Grayscale Frame 2")
plt.axis("off")
```

```
plt.show()
```

Grayscale Frame 1



Grayscale Frame 2



```
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(
    gray1,
    mask=None,
    **feature_params
)

if p0 is not None:
    print("Number of feature points detected:", len(p0))
else:
    print("No feature points detected.")
```

Number of feature points detected: 22

```
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)
```

```

    )
)

p1, st, err = cv2.calcOpticalFlowPyrLK(
    gray1,
    gray2,
    p0,
    None,
    **lk_params
)

good_new = p1[st == 1]
good_old = p0[st == 1]

print("Tracked feature points:", len(good_new))

```

Tracked feature points: 22

```

lk_output = frame2.copy()

for new, old in zip(good_new, good_old):
    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    x_new = int(x_new)
    y_new = int(y_new)
    x_old = int(x_old)
    y_old = int(y_old)

    # Draw motion arrow
    cv2.arrowedLine(
        lk_output,
        (x_old, y_old),
        (x_new, y_new),
        (255, 0, 0),
        2,
        tiplength=0.3
    )

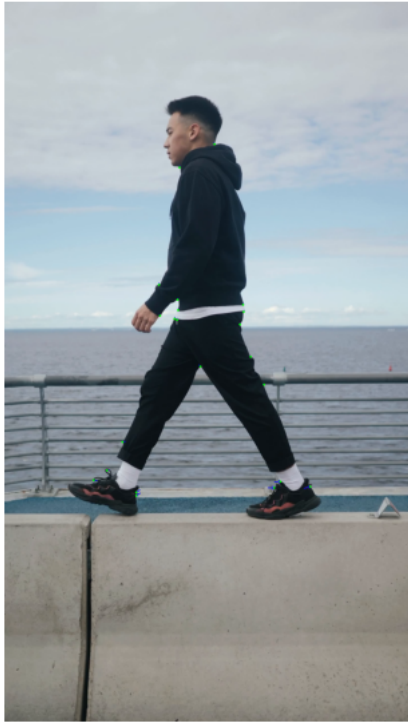
    # Draw tracked point
    cv2.circle(
        lk_output,
        (x_new, y_new),
        4,
        (0, 255, 0),
        -1
    )

lk_rgb = cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(12, 7))
plt.imshow(lk_rgb)
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")
plt.show()

```

Lucas-Kanade Sparse Optical Flow



```
flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0
)

print("Farneback Dense Optical Flow calculated successfully!")
print("Flow shape:", flow.shape)
```

```
Farneback Dense Optical Flow calculated successfully!
Flow shape: (2560, 1440, 2)
```

```
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

print("Motion magnitude calculated.")
print("Average motion magnitude:", np.mean(magnitude))
print("Maximum motion magnitude:", np.max(magnitude))
```

```
Motion magnitude calculated.
Average motion magnitude: 2.6584623
Maximum motion magnitude: 61.400154
```

```
hsv = np.zeros_like(frame1)

hsv[..., 1] = 255

# Direction
hsv[..., 0] = angle * 180 / np.pi / 2

# Magnitude
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

dense_flow = cv2.cvtColor(
```

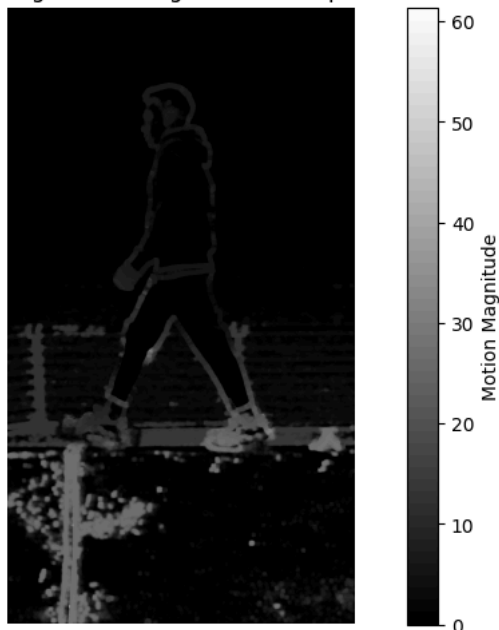
```
hsv,\n    cv2.COLOR_HSV2RGB\n)\n\nplt.figure(figsize=(12, 7))\nplt.imshow(dense_flow)\nplt.title("Farneback Dense Optical Flow")\nplt.axis("off")\nplt.show()
```

Farneback Dense Optical Flow



```
plt.figure(figsize=(10, 6))\n\nplt.imshow(magnitude, cmap='gray')\nplt.colorbar(label="Motion Magnitude")\nplt.title("Motion Magnitude using Farneback Optical Flow")\nplt.axis("off")\n\nplt.show()
```

Motion Magnitude using Farneback Optical Flow



```
comparison = {\n    "Feature": [\n
```

```

        "Type",
        "Motion Estimation",
        "Points/Pixels",
        "Computational Cost",
        "Output",
        "Typical Use"
    ],

    "Lucas-Kanade": [
        "Sparse Optical Flow",
        "Selected feature points",
        "Selected points only",
        "Lower",
        "Motion vectors/trajectories",
        "Object tracking"
    ],

    "Farneback": [
        "Dense Optical Flow",
        "Motion at every pixel",
        "Every pixel",
        "Higher",
        "Dense color-coded flow",
        "Motion analysis"
    ]
}

import pandas as pd

df = pd.DataFrame(comparison)
df

```

	Feature	Lucas-Kanade	Farneback
0	Type	Sparse Optical Flow	Dense Optical Flow
1	Motion Estimation	Selected feature points	Motion at every pixel
2	Points/Pixels	Selected points only	Every pixel
3	Computational Cost	Lower	Higher
4	Output	Motion vectors/trajectories	Dense color-coded flow
5	Typical Use	Object tracking	Motion analysis

```

cap = cv2.VideoCapture(video_path)

ret, old_frame = cap.read()

if not ret:
    print("Unable to read video.")
else:
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        maxCorners=100,
        qualityLevel=0.3,
        minDistance=7,
        blockSize=7
    )

    frame_number = 0

    while True:
        ret, frame = cap.read()

        if not ret:
            break

        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        if p0 is not None:
            p1, st, err = cv2.calcOpticalFlowPyrLK(
                old_gray,
                frame_gray,
                p0,
                None,
                **lk_params
            )

```

```

    )

    if p1 is not None:
        good_new = p1[st == 1]
        good_old = p0[st == 1]

        for new, old in zip(good_new, good_old):
            x_new, y_new = new.ravel()
            x_old, y_old = old.ravel()

            x_new = int(x_new)
            y_new = int(y_new)
            x_old = int(x_old)
            y_old = int(y_old)

            cv2.arrowedLine(
                frame,
                (x_old, y_old),
                (x_new, y_new),
                (255, 0, 0),
                2
            )

            cv2.circle(
                frame,
                (x_new, y_new),
                4,
                (0, 255, 0),
                -1
            )

        p0 = good_new.reshape(-1, 1, 2)

    old_gray = frame_gray
    frame_number += 1

    # Display every 10th frame
    if frame_number % 10 == 0:
        display_frame = cv2.cvtColor(
            frame,
            cv2.COLOR_BGR2RGB
        )

        plt.figure(figsize=(10, 6))
        plt.imshow(display_frame)
        plt.title(f"Lucas-Kanade Optical Flow - Frame {frame_number}")
        plt.axis("off")
        plt.show()

    # Stop after 50 frames for Colab
    if frame_number >= 50:
        break

cap.release()

print("Video processing completed.")

```

Lucas-Kanade Optical Flow - Frame 10

